PROGRAM NO – CO5-2

Aim: To write a Java program to implement a simple calculator using AWT components.

ALGORITHM

1 . Define the GUI Components:

- Create a class that extends "Frame" and implements "ActionListener".
- Define a TextField "tInput" for input and an array of Button objects "btn[ ]" for the calculator buttons.
- Initialize a String array "btnString[]" with values "7, 8, 9, +,4, 5, 6,-, 1, 2, 3, *, C, 0, =, / "
- Initialize variables "num1,num2, result" to 0 for storing input numbers.

2 .Initialize the GUI:

- Define constructor "Calculator".
- Define TextField "tInput" and buttons with corresponding captions.
- Create a panel and set its layout to a 4x4 grid.
- Add the text field to the top of the frame and the panel to the center.
- Add buttons to the panel and register them for action events.

3 . Handle Button Clicks:

- Implement the "actionPerformed" method to handle button clicks.
- Retrieve the clicked button's label using "getActionCommand()".
- If the button is an operator ("+", "-", "*", "/"), set the operator and store the first numbe
- If the button is "=", calculate the result based on the stored operator and numbers.
- If the button is "C", clear the input field and reset variables.
- Otherwise, append the button label to the input field.

4 . Perform Calculation:

- When the "=" button is clicked, perform the calculation based on the stored operator a numbers.
- Use a switch statement to handle addition, subtraction, multiplication, and division.
- Update the input field with the result.

5 . Handle Clear Button:

- When the "C" button is clicked, clear the input field and reset all variables.

Scanned with OKEN Scanner

6 . Main Method:

- Create an instance of the "Calculator" class.
- Set the title and size of the frame.
- Make the frame visible.

PROGRAM

```
import java.awt.*;
import java.awt.event.*;
class Calculator extends Frame implements ActionListener // Frame (a top-level window) and implements
ActionListener (to handle button clicks).

{

  TextField tInput;      // Input/output field for showing numbers and results.

  Panel panel;                  // container to hold calculator buttons
  String btnString[] = {"7", "8", "9", "+",    // Array of button labels (0–9, operations, clear, equals)
              "4", "5", "6", "-",
              "1", "2", "3", "*",
              "C", "0", "=", "/"};
  Button btn[] = new Button[16];       // Array to hold 16 buttons

  int num1 = 0, num2 = 0, result = 0;    // num1,num2 -Operands for calculations

                                  //result- stores the result of the calculation.

  char op;                        //op- Stores the selected operation (+, -, *, /).
  public Calculator()             // Constructor - Sets up the calculator's GUI
  {
    tInput = new TextField(10);       // Creates a text field and a panel
    panel = new Panel();
    add(tInput, "North");             // Places the text field at the top (North) and the panel in the center
    add(panel, "Center");
    panel.setLayout(new GridLayout(4,4));  // Arranges buttons in a 4×4 grid
    for(int i=0; i < 16; i++)
    {
      btn[i] = new Button(btnString[i]);       // Creates each button, sets its label
      btn[i].addActionListener(this);                 // Adds an event listener
      panel.add(btn[i]);                       //Adds each button to the panel.
    }
    addWindowListener(new WindowAdapter()      // Window Close Handling
    {
      public void windowClosing(WindowEvent we)          //Closes the application when the window is closed
      {
```

Scanned with OKEN Scanner

```
            System.exit(0);
        }
    });
}
public void actionPerformed(ActionEvent ae)                    // Button Click Handler-Handles all button clicks.
{
    String str = ae.getActionCommand();                 // retrieves the label (text) of the button that was clicked.
    /* if(str.equals("+"))
    {
        op = '+';
        num1 = Integer.parseInt(tInput.getText());      // Stores the 1st operand (num1) & operator (op)
        tInput.setText("");                             // Clears the input field
    }
    else if(str.equals("-"))
    {
        op = '-';
        num1 = Integer.parseInt(tInput.getText());
        tInput.setText("");
    }
    else if(str.equals("*"))
    {
        op = '*';
        num1 = Integer.parseInt(tInput.getText());
        tInput.setText("");
    }
    else if(str.equals("/"))
    {
        op = '/';
        num1 = Integer.parseInt(tInput.getText());
        tInput.setText("");
    } */
if (str.equals("+") || str.equals("-") || str.equals("*") || str.equals("/"))
    {
            op = str.charAt(0);             // Get 1st character of the string str & store it in the variable op.
            num1 = Integer.parseInt(tInput.getText());    // Stores the 1st operand (num1) & operator (op)
            tInput.setText("");                           // Clears the input field
    }
    else if(str.equals("="))
    {
        num2 = Integer.parseInt(tInput.getText());      // Retrieves 2nd operand (num2)
```

Scanned with OKEN Scanner

```
        switch(op)                                  // performs calculation based on op
        {
          case '+' : result = num1 + num2;
                break;
          case '-' : result = num1 - num2;
                  break;
          case '*' : result = num1 * num2;
                  break;
          case '/' : result = num1 / num2;
                  break;
        }
        tInput.setText(result + "");                //displays result in tInput TextField
        result = 0;
      }
      else if(str.equals("C"))                  // if Clear Button (C)

     {
        tInput.setText("");
        num1 = num2 = result = 0;                   // Clears input and resets all values
      }
      else
      {
        tInput.setText(tInput.getText() + str);     // Appends the digit to the current text
      }
    }
      public static void main(String args[])
      {
        Calculator m = new Calculator();            // Creates the calculator window, sets title, size, and makes
it visible.
        m.setTitle("My Calculator");
        m.setSize(250,300);
-        m.setVisible(true);
      }
    }
```

OUTPUT

Scanned with OKEN Scanner

Java File Handling

- java.io package provides support for input and output (I/O) operations in Java.
- All these streams represent an input source and an output destination.

Stream

- A stream can be defined as a sequence of data.

2 kinds of Streams

1) InPutStream - The InputStream is used to read data from a source.

2)OutPutStream - The OutputStream is used for writing data to a destination.

Byte Streams

- Java byte streams are used to perform input and output of 8-bit bytes.
- Most frequently used classes are
  - FileInputStream
  - FileOutputStream

Character Streams

- Java Character streams are used to perform input and output for 16-bit unicode.
- Most frequently used classes are
  - FileReader
  - FileWriter. Though internally FileReader uses
- FileInputStream and FileWriter uses FileOutputStream, the major difference is that

Scanned with OKEN Scanner

FileReader reads two bytes at a time and FileWriter writes two bytes at a time.

File Operations in Java

- In Java, a File is an abstract data type.
- A named location used to store related information is known as a File.
- The File class from the java.io package, allows us to work with files.

File Operations

1. Create a File
2. Get File Information
3. Write to a File
4. Read from a File
5. Delete a File

1.Create a File

- Create a File operation is performed to create a new file.
- We use the createNewFile() method of file.
- The createNewFile() method returns true when it successfully creates a new _le and returns false when the file already exists.

2. Get File Information

- The operation is performed to get the file information.
- We use several methods to get the information about the file like name, absolute path, is readable, is writable and length.

3. Write to a File

- In order to write data into a file, we will use the FileWriter class and its write( ) method together.
- We need to close the stream using the close( ) method to retrieve the allocated resources.

4.Read from a File

- In order to read data from a file, we will use the Scanner class.
- We will create an instance of the Scanner class and use the hasNextLine() method nextLine() method to get data from the file.
- Here, we need to close the stream using the close() method.

5. Delete a File

- In order to delete a file, we will use the delete() method of the file.
- We don't need to close the stream using the close( ) method because for deleting a file, we neither
  use the FileWriter class nor the Scanner class.

Scanned with OKEN Scanner

PROGRAM NO – CO6-1

Aim: To write a Java program to write to a file, then read from the file and display the contents on the console.

ALGORITHM

1.Start
2.Import necessary Java I/O classes: BufferedReader, FileReader, FileWriter, IOException
3. Define a public class named " FileWrite ".
4. Inside the "FileWrite" class, define the "main" method as the entry point for the program
5. Enclose the code within a try-catch block to handle any potential I/O exceptions.
6. Create a Scanner object to read values from user.
7. Read name of file into "fname".
8. Create a FileWriter object "obW" to write to a file name stored in fname.
9. Read number of entries to be entered into file as "n".
10. Input data to be written into file.
11. Write that data to the file using the write method
12. Close the FileWriter object "obW" to release system resources.
13. Create a FileReader object "obR" to read from the same file
14. Wrap the FileReader object "obR" in a "BufferedReader" for efficient reading:
15.Using while loop, read each line from the file until end of file (-1 is returned) is reached.
16. Print the data read from the file.
17. Close the FileReader object "obR".
18. Catch any "IOException" that might occur during file operations and print the message "-----Error-----".
19.Stop

PROGRAM

```
import java.io.BufferedReader;          // BufferedReader, FileReader, FileWriter: For reading from and writing to files.
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;     // To handle file-related errors.
import java.util.Scanner;       // For reading user input from the console

public class FileWrite
{
   public static void main(String[] args)
   {
      try
      {
         Scanner s= new Scanner(System.in);
         System.out.print("\nEnter File Name : ");
         String fname = s.nextLine();

         FileWriter obW = new FileWriter(fname,true); // Creates a FileWriter object "obW" to write to  specified file.
                                                      // "true" means append mode, so new data is added at the
                                                         end of the file without erasing existing content
         System.out.print("\nEnter no. of entries in file : ");
         int n = s.nextInt();
         System.out.println("\nEnter "+n+" entries into file ");
         String data;
```

Scanned with OKEN Scanner

```
        for(int i=0;i<=n;i++)
        {
            data = s.nextLine();
            obW.write(data + "\n");
        }

        obW.close();            // Close FileWriter- Ensures data is saved and file is properly closed.

         // Read File and Display Contents
        FileReader obR = new FileReader(fname);          // Opens the same file for reading.
        BufferedReader br= new BufferedReader(obR);
        String line;
        System.out.println("Data read from "+fname+" file");
        while ((line = br.readLine()) != null)              // Reads and prints each line 1 by 1
        {
            System.out.println(line);

        }
        obR.close();
    }
    // Error Handling
    catch (IOException e)
    {
        System.out.println("----Error ---- ");     // Catches file-related errors like "file not found", permission issues,
etc.
    }
  }
}
```

Scanned with OKEN Scanner

PROGRAM NO – CO6-2

Aim: To write a Java program to program to copy one file to another.

ALGORITHM

1. Start
2. Import necessary Java I/O classes: FileInputStream, FileOutputStream, IOException
3. Enclose the code within a try-catch block to handle any potential I/O exceptions.
4. Open the input file ("BCA.txt") for reading using FileInputStream.
5. Open the output file ("MCA.txt") for writing using FileOutputStream.
6. Read bytes from the input file "BCA.txt" using the read( ) method of FileInputStream until the end of
   the file is reached (-1 is returned).
7. Write each byte read from the input file "BCA.txt" to the output file "MCA.txt" using the write() method of FileOutputStream.
8. Close both input and output streams using the close() method.
9. Print a success message indicating that the file has been copied successfully.
10. Catch any "IOException" that might occur during file operations and print the message "----- Error-----".
11. Stop

PROGRAM

```
import java.io.FileInputStream;                    // Used for reading from & writing to files as a stream of bytes.

import java.io.FileOutputStream;

import java.io.IOException;                        // IOException handles input/output errors (like file not found).

public class FileCopy

{

  public static void main(String[] args) throws IOException  // throws IOException clause means the method doesn't

  {                                                //.....handle file exceptions internally – any I/O errors will be passed to the JVM.

   try

   {

     FileInputStream fi = new FileInputStream("BCA.txt");       // Opens BCA.txt for reading.

     FileOutputStream fo = new FileOutputStream("MCA.txt"); // Opens MCA.txt for writing. If MCA.txt does not

                                                        //.....exist, it will be created. If it does exist, it will be overwritten.
```

Scanned with OKEN Scanner

```
    int i;
    // Read from Source and Write to Destination
    while ((i = fi.read()) != -1)  // Reads one byte at a time from "fi" (BCA.txt), read() method returns -1 when the end of the
    {                              //.....file is reached
        fo.write(i);               // writes it to "fo" (MCA.txt).
    }
    System.out.println("Successfully copied one file to another");
    fi.close();            // Closes both streams to free system resources.
    fo.close();
  }
  catch (IOException e)
  {
    System.out.println("Error: " + e.getMessage());
  }
 }
}
```

NOTE

- Before executing program, create 2 notepad files manually in the same location where your Java programs are saved.
- Enter different set of data in 2 notepad files and save it.
- Execute the program

OUTPUT

Successfully copied one file to another

NOTE

- Now open the 2nd file & you will find that contents of 1st file is copied to 2nd file.

Scanned with OKEN Scanner

Networking: Socket Programming in Java

- Classes, methods and interfaces required for networking are defined in java.net package.
- The basic concept of networking is a socket. A socket defines an end point in a network.
- A numbered socket on a particular machine is referred to as a port.
- A server "listens" to a port until a client connects to it.
- In networking, a **port number** identifies a specific **process or service** running on a computer.

Networking Protocols

- Communication occur via a protocol.
- Internet Protocol (IP) is a set of rules that dictate how data should be delivered over the public network (Internet). Often works in conjunction with the Transmission Control protocol (TCP), which divides traffic into packets for efficient transport through the Internet; together they are referred to as TCP/IP.
- Another way to use IP is with Datagrams. UDP/IP (User Datagram Protocol) is considered to be

  connection-less, fast and packet loss is not a major issue.

Establishing Client Server Connection

Communication using TCP/IP

- TCP is one of the most used protocols in digital network communications and is part of the Internet protocol suite, commonly known as the TCP/IP suite.
- TCP ensures end-to-end delivery of data between distinct nodes. TCP works in collaboration with Internet Protocol, which defines the logical location of the remote node, whereas TCP transports and ensures that the data is delivered to the correct destination.
- **Before transmitting data, TCP creates a connection between the source and destination node and keeps it live until the communication is active.**
- TCP breaks large data into smaller packets and also ensures that the data integrity is intact

  once it is reassembled at the destination node.

- In order to establish this client server communication, Java uses **two types of sockets.**

  1. Server Socket waits for the client requests (when a client makes a new Socket( ))

  2. Client Socket: software object that:

Scanned with OKEN Scanner

- **Initiates a connection** to a server over a network (usually via TCP),
- **Sends and receives data** over that connection,
- Acts as the **"client"** in the client-server communication model.

TCP Server

- Server application makes a ServerSocket object on a specific port say, 7011.
- This starts our Server listening for client requests coming in for port 7011.
- Then Server makes a new Socket to communicate with the client.

```
socket = server.accept()
```

- The accept() method blocks(just sits there) until a client connects to the server.
- Then we take input from the socket using getInputStream() method.
- To communicate over a socket connection, streams are used to both input and output the data.

TCP Client

- To connect to other machine we need a socket connection. A socket connection means the two machines have information about each other's network location (IP Address) and TCP port.
- To open a socket:

```
Socket socket = new Socket(\127.0.0.1", 7011)
```

First argument -IP address of Server.

Second argument - TCP Port. (Just a number representing which application to run on a server)

Scanned with OKEN Scanner

Communication using socket -TCP/IP : Program Explanation

- **MyServer.java** creates a server socket that listens on port **7011**.
- **MyClient.java** connects to that port on **localhost**, sends the string **"Client Call!!!"**, then exits.

Localhost *is the nickname for your own computer when it talks to itself over the network stack.*

Purpose: *lets you test or run network programs locally without any physical network connection.*

So when a program connects to localhost, it's really connecting to a service running on the same

device.

- The server accepts the connection, reads the UTF-encoded string, prints it, and then closes.

UTF(Unicode Transformation Format)-encoded string*- When Java code uses*

DataOutputStream.writeUTF() *and* ***DataInputStream.readUTF()****, the string is encoded in a very*

specific, Java-defined format called ***"modified UTF-8."***

Run order:

1. Compile both files

   javac MyServer.java

   javac MyClient.java

2. Start the server in one terminal

   java MyServer

3. Start the client in another terminal

   java MyClient

   Output on the server side:

   message= Client Call!!!

Scanned with OKEN Scanner

PROGRAM NO – CO6-3

Aim: To write a Java program to implement client-server communication using socket – TCP/IP.

ALGORITHM

I . Server Side:

1. Create a "ServerSocket" instance and specify the port number (7011 in this case).
2. Use the "accept()" method to wait for an incoming connection request from a client.
3. Once a connection is established, create a DataInputStream object "dis" to read data from the client.
4. Read the UTF-encoded string sent by the client using "readUTF()" method.
5. Process the received message (in this case, print it to the console).
6. Close the "Server Socket".

II. Client Side:

1. Create a "Socket" instance and specify the server's address (localhost) and port number (7011).
2. Create a DataOutputStream "dout" to send data to the server.
3. Write the UTF-encoded string message ("Client Call!!!") to the server using "writeUTF()" method.
4. Flush the output stream to ensure all data is sent.
5. Close the output stream and socket after sending the message.

PROGRAM

```
//MyServer.java

import java.io.*;

import java.net.*;

public class MyServer

{
```

Scanned with OKEN Scanner

```
  public static void main(String[] args)
  {
    try
    {
      ServerSocket serverSocket=new ServerSocket(7011); // Opens TCP port 7011 and starts listening.
      Socket socket=serverSocket.accept();  // Blocks until a client connects, then returns a Socket for that connection.
      //Establishes connection
       DataInputStream dis=new DataInputStream(socket.getInputStream()); // Wraps the raw input stream so
                                                                          //you can read Java-encoded primitives/strings.
      String str=(String)dis.readUTF();  // Reads a UTF-8 string that the client wrote with writeUTF().
      System.out.println("message= "+str); // Prints the received message.
      serverSocket.close(); // Closes both the client socket and the listening socket (program ends).
    }
    catch(Exception e)
    {
      System.out.println(e);
    }
  }
}
```

//MyClient.java

```
 import java.io.*;
import java.net.*;
public class MyClient
{
  public static void main(String[] args)
  {
    try
    {
      Socket socket=new Socket("localhost",7011); // Opens a TCP connection to port 7011 on the same machine.
      DataOutputStream dout=new DataOutputStream(socket.getOutputStream()); // Wraps the output stream for easy
                                                                             //writing of primitives/strings.
      dout.writeUTF("Client Call!!!"); // Sends the length-prefixed UTF string to the server.
```

Scanned with OKEN Scanner

```
    dout.flush(); // Forces any buffered bytes out.
    dout.close();  // Cleanly closes the connection.
    socket.close();
  }
  catch(Exception e)
  {
    System.out.println(e);
  }
}
```

OUTPUT

```
C:\Windows\System32\cmd.exe - java MyServer
Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.
```

Scanned with OKEN Scanner

Communication using UDP/IP

UDP is a communication protocol that transmits independent packets over the network with no guarantee of arrival and no guarantee of the order of delivery.

UDP (User Datagram Protocol) communication in Java.

- Is **connectionless** (no persistent connection),
- Sends **datagrams** (independent packets of data),
- Is **faster but less reliable** (no guaranteed delivery or order).

UDP Server

- In UDP communication, a single message is encapsulated in a DatagramPacket which is sent through a DatagramSocket.
- We create a global DatagramSocket which we will use throughout to send packets, a byte array to wrap our messages, and a status variable called **running**.
- Next, we call the receive method on the socket. This method blocks until a message arrives and it stores the message inside the byte array of the DatagramPacket passed to it.
- After receiving the message, we retrieve the address and port of the client, since we are going to send the response back.
- Next, we create a DatagramPacket for sending a message to the client. There is a difference in signature with the receiving packet. This one also requires address and port of the client we are sending the message to.

Scanned with OKEN Scanner

UDP Client

- We have our global DatagramSocket and address of the server. We instantiate these inside the constructor.
- We have a separate method which sends messages to the server and returns the response. We 1st convert the string message into a byte array, then create a DatagramPacket for sending messages.
- Next we send the message. We immediately convert the DatagramPacket into a receiving one.

Steps of sending and receiving data over UDP are as follows:-

1. For sending a packet via UDP, we should know 4 things,
    - Message to send,
    - its length
    - ip address of destination
    - port at which destination is listening.
2. Once we know all these things, we can create the socket object for carrying the packets and packets

   which actually possess the data.
3. Invoke send( )/receive( ) call for actually sending/receiving packets.
4. Extract the data from the received packet.

Scanned with OKEN Scanner

PROGRAM NO – CO6-4

Aim: To write a Java program to implement client-server communication using datagram socket – UDP.

ALGORITHM

I. Server Side:

1. Create a "DatagramSocket" with a specified port number (e.g., 9000).
2. Prepare a byte array ("buf") to store incoming data.
3. Create a "DatagramPacket" to receive data.
4. Call ` "receive()" method on the socket to wait for incoming packets.
5. Extract the received data from the packet.
6. Process the data as needed.
7. Close the socket when done.

II. Client Side:

1. Create a "DatagramSocket" without specifying a port (a random port will be assigned).
2. Get the IP address of the server using "InetAddress.getByName()".
3. Prepare the data to be sent (in this case, a string message).
4. Convert the data to bytes using "getBytes()".
5. Create a "DatagramPacket" with the data, server address, and port number.
6. Send the packet using "send()" method on the socket.
7. Close the socket when done.

Note:Ensure both client and server are running on the same machine in this example since "localhost" is used. If they are on different machines, replace "localhost" with the server's IP address. Additionally, error handling and more complex logic can be added for robustness and scalability.

Program Explanation

Program Purpose

- **MyClient1** sends a message to port **9000** on localhost.
- **MyServer1** listens on port **9000** and prints the received message.

Concept	Description
DatagramSocket	Used for sending and receiving UDP packets.
DatagramPacket	Represents a single UDP packet (data + address info).
UDP vs TCP	UDP is faster and simpler, but does not guarantee delivery, ordering, or duplicate protection.
Blocking I/O	receive() blocks until a packet arrives.

Scanned with OKEN Scanner

PROGRAM

```
//MyServer1.java - Receiving a UDP message
import java.io.*;
import java.net.*;
public class MyServer1
{
   public static void main(String[] args) throws IOException
   {
      DatagramSocket dSocket=new DatagramSocket(9000); // Opens a socket on UDP port 9000 to
receive data.
      byte[ ] buf=new byte[256];                          // Prepares a buffer and packet object to
      DatagramPacket packet=new DatagramPacket(buf,buf.length); //.... receive up to 256 bytes.
      dSocket.receive(packet);      // Blocks until a datagram arrives, then fills packet with the received
data.
      String response =new String(packet.getData());   // Converts the byte data into a String
      System.out.println(" Server : "+response);          //..... and prints it.
      dSocket.close();    // Closes the socket after receiving one message.
   }
}
//MyClient1.java - Sending a UDP message
import java.io.*;
```

Scanned with OKEN Scanner

```
import java.net.*;
public class MyClient1
{
   public static void main(String[] args) throws IOException
   {
      DatagramSocket dSocket= new DatagramSocket();        // Creates a UDP socket and resolves the destination
      InetAddress add=InetAddress.getByName("localhost");  //..... address (localhost = 127.0.0.1).
      String str ="* Message to Server from Client *";    // Converts the string message into bytes.
      byte[] bufBytes = str.getBytes();
      DatagramPacket datagramPacket=new DatagramPacket(bufBytes,bufBytes.length,add,9000);  // Packages and
      dSocket.send(datagramPacket);    //..... sends the message to the server listening on port 9000.
      dSocket.close();        // Closes the client socket.
   }
}
```

OUTPUT

Scanned with OKEN Scanner